\# Especificación de Casos de Prueba

\## VotaGT - Sistema de Gestión Electoral

Estándar de referencia: ISO/IEC/IEEE 29119-3:2021

Curso: Análisis de Sistemas II

Versión: 1.0

Fecha: Octubre 2026

\---

\# 1. Propósito

El presente documento define los casos de prueba utilizados para verificar las principales funcionalidades del sistema VotaGT.

Los casos están relacionados con:

\- Requisitos SRS.

\- Historias de Usuario.

\- Escenarios BDD.

\- Endpoints OpenAPI.

\- Colección Postman.

\- Pruebas automatizadas.

\---

\# 2. Alcance

Las pruebas cubren:

\- Autenticación.

\- Autorización.

\- Votaciones.

\- Candidatos.

\- Mesas.

\- Resultados.

\- Caché.

\- Exportación.

\- Manejo de errores.

\- Resiliencia.

\---

\# 3. Entorno de prueba

\## Backend

Node.js + Express

URL base:

`http://localhost:3000/api/v1`

\## Persistencia

\- PostgreSQL 18.

\- Redis 7.

\- MongoDB 7.

\- Neo4j 5 Community.

\## Herramientas

\- Postman.

\- Node Test Runner.

\- Swagger Editor.

\- Docker Desktop.

\- PostgreSQL psql.

\- Neo4j Browser.

\---

\# 4. Precondiciones generales

Antes de ejecutar las pruebas:

1\. PostgreSQL debe estar disponible.

2\. La base `votagt` debe estar creada.

3\. El backend debe estar iniciado.

4\. MongoDB y Neo4j deben estar disponibles para las pruebas relacionadas.

5\. Redis debe estar activo excepto durante pruebas de resiliencia.

6\. Debe existir un usuario de prueba con permisos administrativos.

7\. Los endpoints deben estar configurados en Postman.

\---

\# 5. Estados de prueba

Los estados utilizados son:

\- Pendiente.

\- En ejecución.

\- Aprobado.

\- Fallido.

\- Bloqueado.

\---

\# 6. Casos de prueba funcionales

\## CP-01 - Inicio de sesión exitoso

Requisito: REQ-FN-001

Historia: HU-01

Tipo: Funcional positivo

Prioridad: Alta

\### Precondición

El usuario existe y se encuentra activo.

\### Entrada

```json
{
&#x20; "identificador": "admin",
&#x20; "password": "Admin123\*"
}
```

\### Procedimiento

1\. Enviar solicitud `POST /auth/login`.

2\. Enviar credenciales válidas.

3\. Revisar la respuesta.

\### Resultado esperado

\- HTTP 200.

\- Se devuelve un JWT.

\- Se devuelve información del usuario.

\- Se genera evento `LOGIN\_EXITOSO`.

\### Estado

Aprobado.

\---

\## CP-02 - Inicio de sesión con credenciales incorrectas

Requisito: REQ-FN-001

Historia: HU-01

Tipo: Funcional negativo

Prioridad: Alta

\### Entrada

Credenciales incorrectas.

\### Procedimiento

1\. Enviar `POST /auth/login`.
2\. Utilizar contraseña incorrecta.

\### Resultado esperado

\- HTTP 401.
\- No se genera token.
\- El mensaje no revela cuál credencial es incorrecta.

\### Estado

Aprobado.

\---

\## CP-03 - Acceso al perfil sin token

Requisitos: REQ-FN-002, REQ-NF-001
Historia: HU-01
Tipo: Seguridad
Prioridad: Alta

\### Procedimiento

1\. Enviar `GET /auth/perfil`.
2\. No enviar Bearer Token.

\### Resultado esperado

\- HTTP 401.
\- Respuesta `application/problem+json`.

\### Estado

Aprobado.

\---

\## CP-04 - Crear votación correctamente

Requisito: REQ-FN-004
Historia: HU-05
Tipo: Funcional positivo
Prioridad: Alta

\### Precondiciones

\- Usuario autenticado.
\- Permiso para gestionar votaciones.

\### Procedimiento

1\. Enviar `POST /votaciones`.
2\. Incluir nombre.
3\. Incluir fecha inicial.

4\. Incluir fecha final posterior a la inicial.

\### Resultado esperado

\- HTTP 201.

\- La votación queda almacenada en PostgreSQL.

\- Se genera `VOTACION\_CREADA`.

\### Estado

Aprobado.

\---

\## CP-05 - Registrar mesa

Requisito: REQ-FN-009

Historia: HU-06

Tipo: Funcional positivo

Prioridad: Alta

\### Precondiciones

\- Votación existente.

\- Usuario autorizado.

\### Entrada de ejemplo

```json

{
```

  "codigo": "MESA-01",

  "nombre": "Mesa Central",

  "ubicacion": "Centro",

  "departamento": "Guatemala",

  "municipio": "Guatemala"

}

```

\### Resultado esperado

\- HTTP 201.

\- La mesa queda asociada a la votación.

\- Se genera `MESA\_CREADA`.

\### Estado

Aprobado.

\---

\## CP-06 - Registrar candidato

\*\*Requisito:\*\* REQ-FN-007

\*\*Historia:\*\* HU-07

\*\*Tipo:\*\* Funcional positivo

\*\*Prioridad:\*\* Alta

\### Precondiciones
```

\- Votación existente.

\- Usuario autorizado.

\### Entrada de ejemplo

```json
{
&#x20; "nombre": "Candidato Prueba",
&#x20; "partido": "Partido Prueba",
&#x20; "cargo": "Presidente",
&#x20; "numeroLista": 10
}
```

\### Resultado esperado

\- HTTP 201.

\- El candidato queda almacenado.

\- Se genera `CANDIDATO\_CREADO`.

\### Estado

Aprobado.

\---

\## CP-07 - Registrar resultado

Requisito: REQ-FN-010

Historia: HU-08

Tipo: Funcional positivo

Prioridad: Crítica

\### Precondiciones

\- Votación existente.

\- Mesa existente.

\- Candidato existente.

\- Usuario con permiso `RESULTADOS\_REGISTRAR`.

\- No debe existir previamente la misma combinación mesa/candidato.

\### Entrada

```json
{
&#x20; "mesaId": 1,
&#x20; "candidatoId": 1,
&#x20; "votos": 50
}
```

\### Resultado esperado

\- HTTP 201.

\- Resultado almacenado en PostgreSQL.

\- Caché de la votación invalidada.

\- Evento `RESULTADO\_REGISTRADO`.

\### Estado

Aprobado.

\---

\## CP-08 - Consultar resultados

Requisito: REQ-FN-011

Historia: HU-09

Tipo: Integración

Prioridad: Alta

\### Procedimiento

1\. Enviar `GET /votaciones/{votacionId}/resultados`.

2\. Revisar los resultados consolidados.

\### Resultado esperado

\- HTTP 200.

\- Se informa `votacionId`.

\- Se informa total de votos.

\- Se muestran resultados por candidato.

\### Estado

Aprobado.

\---

\## CP-09 - Exportar resultados CSV

Requisito: REQ-FN-013

Historia: HU-10

Tipo: Funcional positivo

Prioridad: Media

\### Solicitud

`GET /votaciones/{votacionId}/resultados/exportar?formato=csv`

\### Resultado esperado

\- HTTP 200.

\- `Content-Type: text/csv`.

\- Se genera contenido CSV.

\- El archivo contiene resultados de la votación.

\### Estado

Aprobado.

\---

\# 7. Casos de prueba negativos y alternos

\## CP-10 - Registrar votos negativos

Requisito: REQ-FN-010

Historia: HU-08

Tipo: Validación negativa

\### Entrada

```json
{
&#x20; "mesaId": 1,
&#x20; "candidatoId": 1,
&#x20; "votos": -10
}
```

\### Resultado esperado

\- HTTP 400.
\- `application/problem+json`.
\- No se almacena el resultado.

\### Estado

Aprobado.

\---

\## CP-11 - Consultar votación inexistente

Requisito: REQ-FN-005

Historia: HU-05

Tipo: Negativo

\### Solicitud

`GET /votaciones/99999`

\### Resultado esperado

\- HTTP 404.
\- `application/problem+json`.

\### Estado

Aprobado.

\---

\## CP-12 - Resultado duplicado

Requisito: REQ-FN-010
Historia: HU-08
Tipo: Conflicto

\### Procedimiento

1\. Registrar un resultado.
2\. Volver a registrar la misma combinación mesa/candidato.

\### Resultado esperado

\- HTTP 409.

\- Tipo de error `duplicate-result`.

\- PostgreSQL no crea un segundo registro.

\### Estado

Aprobado.

\---

\## CP-13 - Exportar resultados JSON

Requisito: REQ-FN-013

Historia: HU-10

Tipo: Funcional positivo

\### Solicitud

`GET /votaciones/{votacionId}/resultados/exportar?formato=json`

\### Resultado esperado

\- HTTP 200.

\- `application/json`.

\- Resultados consolidados disponibles.

\### Estado

Aprobado.

\---

\## CP-14 - Formato de exportación inválido

Requisito: REQ-FN-013

Historia: HU-10

Tipo: Negativo

\### Solicitud

`GET /votaciones/{votacionId}/resultados/exportar?formato=pdf`

\### Resultado esperado

\- HTTP 400.

\- `application/problem+json`.

\- Tipo `invalid-report-format`.

\### Estado

Aprobado.

\---

\# 8. Casos de prueba de caché

\## CP-15 - Cache MISS

Requisito: REQ-NF-004

Historia: HU-09

Tipo: Rendimiento / integración

\### Precondición

No debe existir la clave de resultados en Redis.

\### Procedimiento

1\. Consultar resultados.

2\. Revisar consola del backend.

\### Resultado esperado

```text
CACHE MISS
CACHE GUARDADA
```

La información se obtiene desde PostgreSQL y se almacena temporalmente en Redis.

\### Estado

Aprobado.

\---

\## CP-16 - Cache HIT

Requisito: REQ-NF-004

Historia: HU-09

Tipo: Rendimiento

\### Precondición

La clave debe existir en Redis.

\### Procedimiento

Consultar nuevamente los mismos resultados antes de que expire el TTL.

\### Resultado esperado

```text
CACHE HIT
```

\### Estado

Aprobado.

\---

\## CP-17 - Invalidación de caché

Requisitos: REQ-FN-010, REQ-NF-004

Historia: HU-08

\### Procedimiento

1\. Consultar resultados.

2\. Confirmar que se creó caché.

3\. Registrar un nuevo resultado.

4\. Revisar consola.

5\. Consultar resultados nuevamente.

\### Resultado esperado

La clave de resultados deberá eliminarse.

La siguiente consulta deberá producir:

```text
CACHE MISS
```

\### Estado

Aprobado.

\---

\# 9. Casos de prueba de resiliencia

\## CP-18 - Redis no disponible

Requisitos: REQ-NF-005, REQ-NF-006

Tipo: Resiliencia

Prioridad: Alta

\### Procedimiento

1\. Mantener Node.js ejecutándose.

2\. Detener Redis.

3\. Consultar resultados repetidamente.

\### Resultado esperado

Se deberán observar:

```text
Redis: DEGRADED
FALLO 1/3
FALLO 2/3
FALLO 3/3
CIRCUIT OPEN
```

La API deberá continuar obteniendo los resultados desde PostgreSQL.

\### Resultado HTTP

`200 OK`

\### Estado

Aprobado.

\---

\## CP-19 - Recuperación de Redis

Requisito: REQ-NF-006

Tipo: Resiliencia

\### Precondición

Circuit Breaker en estado OPEN.

\### Procedimiento

1\. Iniciar nuevamente Redis.

2\. Esperar el periodo configurado.

3\. Consultar resultados.

\### Resultado esperado

```text
CIRCUIT HALF\_OPEN
CIRCUIT CLOSED
```

Redis vuelve a utilizarse normalmente.

\### Estado

Aprobado.

\---

\# 10. Casos automatizados

Las pruebas automatizadas se ejecutan mediante:

```cmd
npm test
```

Las pruebas incluyen:

\- Middleware de permisos.

\- Strategy JSON.

\- Strategy CSV.

\- Formato Strategy inválido.

\- Circuit Breaker CLOSED.

\- Circuit Breaker OPEN.

\- Circuit Breaker HALF_OPEN -> CLOSED.

\- Health.

\- Login.

\- Perfil autenticado.

\- Perfil sin token.

\- Consulta de votaciones.

\- Votación inexistente.

El criterio de aceptación es:

```text
```

fail 0

```

\---

\# 11. Relación con BDD

Los casos de autenticación se relacionan con:

`features/autenticacion.feature`

Los casos de gestión electoral se relacionan con:

`features/gestion\_electoral.feature`

Los escenarios BDD incluyen:

\- Camino feliz.

\- Casos de error.

\- Scenario Outline.

\- Tablas de ejemplos.

\---

\# 12. Resumen de ejecución

| ID | Prueba | Resultado |

|---|---|---|

| CP-01 | Login exitoso | Aprobado |

| CP-02 | Login incorrecto | Aprobado |
```

| CP-03 | Perfil sin token | Aprobado |

| CP-04 | Crear votación | Aprobado |

| CP-05 | Registrar mesa | Aprobado |

| CP-06 | Registrar candidato | Aprobado |

| CP-07 | Registrar resultado | Aprobado |

| CP-08 | Consultar resultados | Aprobado |

| CP-09 | Exportar CSV | Aprobado |

| CP-10 | Votos negativos | Aprobado |

| CP-11 | Recurso inexistente | Aprobado |

| CP-12 | Resultado duplicado | Aprobado |

| CP-13 | Exportar JSON | Aprobado |

| CP-14 | Formato inválido | Aprobado |

| CP-15 | Cache MISS | Aprobado |

| CP-16 | Cache HIT | Aprobado |

| CP-17 | Invalidación | Aprobado |

| CP-18 | Redis degradado | Aprobado |

| CP-19 | Recuperación Redis | Aprobado |

\---

\# 13. Criterio de aceptación general

El conjunto de pruebas se considera conforme cuando:

\- No existen defectos críticos abiertos.

\- Los códigos HTTP corresponden al comportamiento esperado.

\- Los errores controlados usan `application/problem+json`.

\- Las pruebas automatizadas presentan `fail 0`.

\- Las operaciones principales están verificadas mediante Postman.

\- Redis puede fallar sin detener las consultas principales.

\- Existe trazabilidad entre pruebas y requisitos.

\---

\# 14. Conclusión

La estrategia de pruebas de VotaGT combina pruebas funcionales, negativas, integración, seguridad, caché y resiliencia.

Los casos documentados permiten comprobar el comportamiento del backend y proporcionan evidencia trazable desde los requisitos hasta la implementación y validación.